

TEAM NAMA

The Neuron Migration

Reverse-engineering 30 years of insurance logic
into code an auditor can check

Project report
Lifeware Challenge, Neural Wave AI Hackathon

Mohammad Javad Amin Nava Biglar Malihe Ghasemian Emad A

Università della Svizzera italiana (USI), Lugano

October 2026

Contents

1 Our idea in one page **2**

2 The problem **2**
- 2.1 The story . . . 2
- 2.2 What we were given . . . 2
- 2.3 The tasks . . . 3
- 2.4 The rules of the game . . . 3

3 Step 1: understanding the data **3**
- 3.1 One loader for every table . . . 3
- 3.2 Problems in the data . . . 3
- 3.3 Following one policy . . . 4

4 Step 2: our method and tools **4**
- 4.1 The rule finder . . . 4
- 4.2 Convention search . . . 4
- 4.3 Checking each booking on its own (teacher forcing) . . . 4
- 4.4 Comparing exactly . . . 4

5 Step 3: the rules, task by task **4**
- 5.1 t0 Commission . . . 4
- 5.2 t1 Next due dates . . . 4
- 5.3 t2 Projected maturity value . . . 5
- 5.4 t3 and t4: the fund accounting . . . 5
- 5.5 Summary: two systems . . . 6

6 Results **6**
- 6.1 Accuracy . . . 6
- 6.2 Other measures . . . 6

7 Implementation **7**

8 Roadmap **7**
- 8.1 What we did . . . 7
- 8.2 What comes next . . . 7

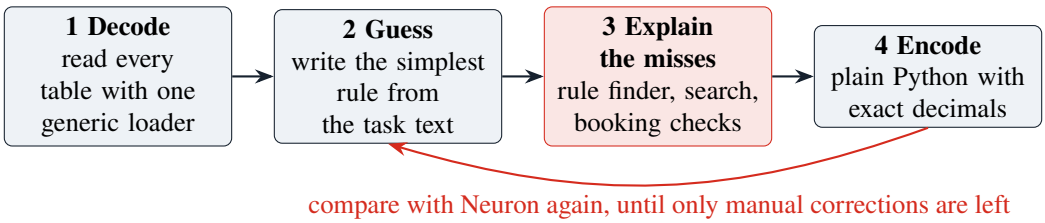
9 Conclusion **8**

A Reproducing the results **8**

1 Our idea in one page

The insurer NeuralWave Life must move its policies from its old system *Neuron* to Lifeware. The new system has to calculate *exactly* the same values as Neuron, to the cent and to the day, but nobody knows how Neuron calculates them: there is no source code, and the documentation is incomplete. We only have Neuron's data and the values it calculated for part of the policies.

Our idea: AI is the detective, code is the answer. A model that *predicts* a surrender value would never be exact, and an auditor cannot check it. So we never put a model into the solution. Instead we use the data and machine learning to *find* Neuron's rules, and we write every rule as short, plain Python with exact decimal numbers. We repeat the same loop for every value Neuron calculates:



What came out of it.

- Every task reaches between 99.7 % and 100 % exact rows on the training policies, and **every remaining difference is a value that was changed by hand**.
- For the surrender values and the annual statements we recalculate the complete fund accounting ourselves: **5,024,847 bookings**, all identical to Neuron's except 3 that were corrected by hand.
- We found **55 hand-edited values**; 17 of them are not in the official list of manual corrections.
- The biggest discovery: Neuron is really **two systems**. The old COBOL system (until August 2007) rounds, counts ages and charges fees differently from the newer Java system, Neuron NG.
- All five tasks run in about three minutes, and the tools we built work for any other target.

2 The problem

2.1 The story

NeuralWave Life is a (fictional) life insurer in Ticino. Since 1994 it has sold *unit-linked* policies: the customer's premiums buy units of investment funds, and the policy is worth the value of these units. For more than 30 years the system Neuron calculated every premium, fee, fund unit and payout. Neuron started in COBOL, was partly rewritten in Java, was patched many times, and sometimes people corrected values by hand. Before Lifeware can take over, it must reproduce Neuron's numbers exactly.

2.2 What we were given

- **The export** (`legacy/`): 27 tables in old formats, for example 4,011 policies with their history, 323,441 premium items, 872,039 business events, 6,797,112 fund unit bookings, fund prices, exchange rates and mortality tables.
- **The targets** (`targets/`): the values Neuron calculated for the 3,019 training policies, plus some diagnostic files (yearly projections, single bookings, fund holdings).
- A partial description of the tables (`SCHEMA.md`) and the task text.

The remaining policies are scored on a public leaderboard and on a private set we never see, so a solution can only win by finding the real rules.

2.3 The tasks

Task	What Neuron calculates
t0 Commission	the commission a broker earns on every premium the customer paid
t1 Next due dates	the premium due dates of the next year and the days the money is collected
t2 Projected maturity value	the value at maturity shown in the offer the customer signed
t3 Surrender value	the full fund accounting, and what the customer gets on surrender
t4 Annual statement	the yearly overview of premiums and all costs
t5 Outliers (bonus)	the values that were changed by hand
t6 Visualisation (bonus)	the tools and pictures that helped to find the rules

2.4 The rules of the game

- A value counts only if it is **exact**: an amount to the cent, a date to the day, all six columns of a statement row.
- The code must calculate everything from the export; it must not read the targets (except for t5).
- The **maintainability** of the code is measured (radon maintainability index), and approaches that **generalise** to new targets earn extra points.

3 Step 1: understanding the data

3.1 One loader for every table

The export uses old formats: blank-padded text, numbers without a decimal point, and dates such as `1250131` for 31 January 2025 (`CYYMMDD`, where C counts centuries from 1900). The data dictionary `NRDICT` gives the type and the number of decimals of every column, even of its own columns. So instead of writing a reader per table, we wrote **one generic decoder** driven by the dictionary:

Type	Raw value	Decoded
C text	`'BL10000001  '`	`'BL10000001'` (blanks trimmed)
N(s) number	`18500` with s = 2	185.00 (s implied decimals)
D date	`1250131`, `970630`	2025-01-31, 1997-06-30; `0` = no date, `9999999` = open end
T timestamp	`20070131154415`	2007-01-31 15:44:15

The decoded tables are cached, so the 400 MB booking table loads in a fraction of a second during exploration. Amounts are converted back to exact decimals before any calculation.

3.2 Problems in the data

Decoding every column with the dictionary already showed what a real migration team would meet:

- 10 **impossible birth dates** such as 31 November 1954; the old system never checked dates.
- The code list `NRCODE` has one text column fewer than its header, so the English texts sit in the German column.
- The smoker flag is `J/N` (German *Ja*), not `Y/N` as documented.
- 80 exchange rates are stored with **lowercase currency codes** (`eur`, `usd`, . . .); they fill days without an uppercase rate and must be read as uppercase.

3.3 Following one policy

Before solving anything we followed one policy (`BL10000013`) through all tables: its versions, premiums, events, bookings and targets. A *policy timeline* tool prints all data of a policy and plots its history. This walk-through already showed several rules, for example that the commission is 3 % of each paid premium and that the first twelve premiums of this product buy "initial" units.

4 Step 2: our method and tools

4.1 The rule finder

For every task we first write the simplest rule. Then we mark the rows where it differs from Neuron and give them to the **rule finder**: a shallow decision tree that learns which conditions separate the wrong rows from the right ones. Its splits are readable, for example "the policy starts on day 29 or later *and* before 2007". A split like this points directly to the hidden rule. The tool knows nothing about insurance, so it works for any target.

4.2 Convention search

Many details are conventions: is a value rounded half up or cut off, is the growth rate divided by 12 or compounded, is the age counted from the last or the nearest birthday, which date decides a parameter? Such questions have few possible answers, so we try **all combinations** and keep the one that reproduces Neuron exactly.

4.3 Checking each booking on its own (teacher forcing)

For the fund accounting every booking depends on all bookings before it, so one small error spoils everything after it. To check a formula on its own, we take the holdings before each booking from Neuron's own (diagnostic) numbers. Then every booking that differs points to exactly one wrong formula.

4.4 Comparing exactly

A value is right only if it is identical. During the project we found that a "close enough" comparison hides one-cent errors on large amounts, so all checks compare to the cent with no relative tolerance.

5 Step 3: the rules, task by task

5.1 t0 Commission

- Only premium items that were **paid** earn a commission; premiums returned by the bank, open items and written-off items earn nothing. (The rule finder found this with one split.)
- The broker is the one of the policy version valid on the **day the money arrived**, so a broker change moves the commission from that day on.
- The rate comes from the commission table by broker tier and premium type (regular or single).
- Commission = received amount × rate, rounded half up to the cent.

5.2 t1 Next due dates

- **Business calendar.** Fund prices are published on every business day, so every weekday without a price is a holiday. They are exactly the public holidays of Ticino: eleven fixed days plus Easter Monday, Ascension, Whit Monday and Corpus Christi (Good Friday is a business day). Written

as a rule, the calendar also works for 2027 and matches 33 years of price dates without a single difference.

- **Due dates.** The old COBOL system added one month to the *previous* due date, so a policy that started on the 31st drifted to the 30th and then stayed on the 28th forever. Neuron NG adds whole months to a fixed anchor; for converted policies the anchor is the last COBOL due date.
- **Collection.** A direct debit is collected on the next business day, a bank transfer on the due date.

5.3 t2 Projected maturity value

The offer simulates the policy month by month with the parameters valid on the **application date**:

1. add the premium, less the entry charge;
2. take the admin fee, then the management charge on what is left, then the risk charge on what is left;
3. let the value grow by the illustration rate divided by 12.

The risk charge is the sum at risk × death probability × (1 + medical loading) × smoker factor ÷ 12, with the age fixed at the start of each policy year. Two discoveries made the last offers exact:

- Neuron NG takes the age at the **nearest birthday** and rounds half up; the COBOL system **truncated** every amount.
- For the last 21 COBOL offers the person was "one year too old". COBOL computed the age as **days since birth ÷ 365**, ignoring leap days. Of several possible formulas, only this one raises the age for exactly these 21 offers and for no other.

5.4 t3 and t4: the fund accounting

The new system must calculate the premium and charge bookings itself. Our engine generates every booking from the premiums, the parameters and the market data; only the customer orders (switches, withdrawals, surrenders, death benefits) are taken from the export, as the task allows.

Premium bookings.

- For each fund of the allocation: received amount × share × (1 − entry charge). Allocation, broker tier, entry charge and individual agreements are all taken on the money date.
- Premiums received in the first months of the policy (12 or 24, by product) buy initial units (pot 1), later ones base units (pot 2).
- Each fund buys on its **own first day with a price** after the money date, at the issue price, converted with the exchange rate of the day before. When one fund has no price, only that fund trades later.

Monthly charges.

- The charge run at 01:30 sees only the units traded **before** that day.
- Every fund and pot is valued at the redemption price and rounded to the cent.
- The admin fee and the risk charge are split over the funds in proportion to their value; the largest position takes the rounding difference, and a share of 0.00 creates no booking.
- The management charge is value × pot rate ÷ 12; the COBOL system charged by the day, rate × days ÷ 360.
- The risk charge uses the value left after the admin fee and the management charge, exactly like the offer.
- Reversed events and technical corrections are ignored, and there are no charges from the day a policy is surrendered or the insured person dies.

Surrender value (t3). The units are valued at the redemption price on or before the valuation date. The penalty depends on the product's surrender scale and the completed policy years, and there is no penalty in the waiver periods (financial crisis 2008, COVID 2020). The COBOL system cut off the surrender value itself.

Annual statement (t4). The statement adds up the bookings by calendar year: premiums by money date, entry charges (premium share less the amount invested) and the monthly charges by trade date.

5.5 Summary: two systems

	COBOL (until 2007-08-11)	Neuron NG (Java)
Rounding	cut off (truncate)	half up
Age of the insured person	days since birth ÷ 365, rounded	nearest birthday
Management charge	rate × days ÷ 360	rate ÷ 12
Due dates	drift after short months	anchored, no drift

6 Results

6.1 Accuracy

Scored with the official scoring on the training policies:

Task	Exact rows	Accuracy	Hand-edited	not in NREXCP
t0 Commission	225,848 / 225,858	99.996 %	10	4
t1 Next due dates	15,028 / 15,028	**100 %**	0	0
t2 Projected maturity value	3,011 / 3,019	99.735 %	8	3
t3 Surrender value	6,137 / 6,153	99.740 %	16	5
t4 Annual statement	34,124 / 34,145	99.939 %	21	5
Total			55	17

Every row that is still different is a manual correction: the amounts differ by whole francs or follow no rule, while everything around them (the yearly projection, the policy value, the other bookings) is exact. The official evaluation does not score manual corrections, so on the scored rows our rules are exact.

6.2 Other measures

- **Fund accounting**: 5,024,847 premium and charge bookings calculated, all identical to Neuron's except three risk bookings of one policy that were corrected by hand.
- **Runtime**: all five tasks together take about three minutes on the full export.
- **Maintainability index**: 70.8 (radon, weighted over all files).

7 Implementation

The code follows one principle: *one small module per task*, plus shared building blocks.

File	Content
`submission.py`	entry point, one function per task
`legacy_loader.py`, `legacy_formats.py`	reading and decoding the export with the data dictionary
`rules.py`	shared rules: date-valid parameter lookups, rounding by system era, conversion date
`business_calendar.py`	business days and Ticino holidays
`commission.py`	t0
`schedule.py`	t1
`policies.py`, `risk.py`, `projection.py`	t2 (risk charge shared with t3)
`market.py`, `premiums.py`, `charges.py`, `accounting.py`	the fund accounting
`surrender.py`, `statement.py`	t3 and t4
`tools/rule_finder.py`	decision tree that explains where a rule is wrong
`tools/policy_timeline.py`	all data and the timeline of one policy
`notebooks/01-04`	data processing and the evidence for every rule

Amounts are `Decimal` throughout, and no date or rate of this export is written into the code (only the last statement year 2025, which the task defines): the conversion date comes from the conversion events, the extract date from the last event, the holidays from rules.

8 Roadmap

8.1 What we did

Phase	Status	Work
1 Understand the data	done	generic loader, data quality findings, walk-through of one policy, timeline tool
2 First tasks	done	t0 and t1; business calendar; rule finder
3 Offer projection	done	t2; convention search; COBOL age
4 Fund accounting	done	t3 and t4; booking engine checked booking by booking
5 Communication	done	four evidence notebooks, this report, a 4-minute presentation

8.2 What comes next

1. **t5 Outliers.** Our calculated values are exact except for the hand-edited ones, so the outlier report is the list of keys where the given targets differ from our values. The 55 differences we already know are the test of this function.
2. **Leaderboard.** Submit the solution and confirm the results on the public policies.
3. **Speed.** The premium bookings still look up policy data row by row; precomputing these lookups should cut the runtime noticeably.
4. **Maintainability.** Split the largest modules further (charge run, market data) and keep every function short; the index weighs every file by its size.

5. **Generalisation.** Turn the convention search into a reusable tool next to the rule finder: given a target and candidate conventions, it reports the combination that matches exactly.
6. **Visualisations (t6).** Add plots of the differences over time and per system era, and a waterfall of an annual statement, to the notebooks.

9 Conclusion

Starting from data alone, we rebuilt Neuron's calculations so that every value matches to the cent, except the values people changed by hand, which we can now name. The key was to use machine learning and search as tools to *find* rules, never to replace them: the result is short, deterministic code that a developer can maintain and an auditor can check. The same loop of decoding, guessing, explaining the misses and encoding can be applied to the next legacy system.

A Reproducing the results

```
python3 -m venv .venv
.venv/bin/pip install -r starter_kit/my_solution/requirements.txt
cd starter_kit && ../.venv/bin/python check.py
```

The challenge data (`legacy/` and `targets/`) must be placed next to the repository's README.